One delayed order in C++

Study: October 23; integrated October 24, 2026. L002c: one delayed-order trace. Sidiq review status: Not yet reviewed. All inputs and results are synthetic teaching examples.

Research question and hypothesis

Can a decision leave cash and shares unchanged while a later accepted fill applies the correct price and fee? We predict that observing 50 at event 0 requests two shares at event 1 without changing 800 cash and four shares. An assumed fill at 52 at event 2 should leave cash 693 and six shares. Missing fills and model rejections should preserve state and charge no fee.

L001 supplied checked accounting; L002a/b defined the timeline and contract. Those earlier lessons were integrated today with their true study dates. Today's one concept is separating the decision snapshot from the resolution state in C++.

Protocol: compare six contract cases and two extra hand anchors with a separate Decimal reference; reject fifteen invalid integer traces; build twice and run each executable twice; detect deliberately wrong source copies. Metrics are exact states/statuses, unchanged caller state and reproducibility. No historical sample, fitted rule or return estimate is used.

Prerequisite recap and terminology

AccountState groups cash_units and shares. Passing initial by value copies the caller's account. Existing try_buy takes a reference and may update the state supplied to it after its guards pass. Return sends a result back; it does not automatically change the caller.

A snapshot saves state at one event. DelayedTrace groups a validity flag, requested quantity, two AccountState copies, status text and fee charged. std::string stores the status text. No enum, header interface, vector or queue is introduced. A false validity flag means the whole trace is invalid; request -1 is a sentinel, never a short sale or valid decision.

Unresolved means no fill evidence at the fixture horizon. Rejected_by_model means this ledger's cash/capacity check failed, not that an exchange rejected an order.

Expected hand outcomes

Default observation/decision/resolution events are 0/1/2; observed price is 50 and fee is 3. Each row starts a separate account. C is cash; q is shares; final means after resolution.

Case	Initial C / q	Fill	Status	Final C / q
Ordinary buy	800 / 4	52	filled	693 / 6
Missing evidence	800 / 4	absent	unresolved	800 / 4
Exact cash	107 / 4	52	filled	0 / 6
Too little cash	106 / 4	52	rejected_by_model	106 / 4
Observe 51	800 / 4	52	no_order	800 / 4
Position cap	800 / 9999	52	rejected_by_model	800 / 9999
51; no fill	800 / 4	absent	no_order	800 / 4
Exact capacity	107 / 9998	52	filled	0 / 10000

Read the decision function slowly

The short decide_quantity function takes only observed_price, available_event and decision_event. It has no future fill argument and does not receive an account that it could debit. Its signature returns int: a requested count, with -1 reserved for an invalid input. This is a teaching interface.

The first if rejects prices outside 1..10000, negative availability, and availability at or after the decision. The expression available_event >= decision_event rejects both equality and a later observation. The || operator means any one failing condition is enough. return -1 exits early.

The second if asks whether observed_price <= 50. If true, return 2 requests two shares. Otherwise return 0 means a valid decision to request nothing. Zero and invalid are deliberately distinct. The literal threshold 50 is a fixed classroom rule, not an estimated profitable signal. Braces group each branch, and the semicolon ends each return statement.

Read the complete trace in order

The delayed_order function accepts an AccountState by value, observation/event integers, a has_fill Boolean, and a fill_price integer. Its return type is DelayedTrace. The first statement initializes invalid status, request -1, both snapshots equal to initial, and fee zero. Returning this object on a bad input preserves the provided fields; it does not turn bad input into a trade.

The first guard rejects cash/shares outside 0..10000, resolution at or before the decision, and a supplied fill price outside 1..10000. The expression has_fill && (...) applies the price check only when the price is present. If has_fill is false, the accompanying price integer is ignored. The Python harness translates JSON null to false plus a harmless zero placeholder. A supplied zero is different: true plus zero is invalid, even when no order would be requested.

The next statement calls decide_quantity with the three known observation fields. A negative result returns the still-invalid trace. Otherwise valid becomes true. The at_decision copy is never changed after initialization. It retains the original cash and shares for all valid cases.

If request is zero, status becomes no_order. Else if !has_fill, status becomes unresolved. Here ! means Boolean negation. Otherwise the code calls the existing try_buy with trace.after_resolution, trace.request, fill_price and fee 3. Only that second snapshot is passed by reference. If the buy succeeds, status becomes filled and fee_charged becomes 3; otherwise status becomes rejected_by_model and the checked buy preserves the state. Finally return trace sends the complete record back. A caller would have to explicitly adopt its final state.

Mathematics and worked example

Use whole money units for cash C and fee f, whole shares for q and requested quantity n, and money per share for price p. At decision: C_decision = C and q_decision = q. At an accepted fill: C_final = C - n*p - f and q_final = q+n. At rejection or missingness, C_final = C and q_final = q; f_charged = 0. Every debit has money units.

For C=800, q=4, n=2, p=52 and f=3: notional is 104, total debit is 107, final cash is 693, and final shares are 6. Cash exactly 107 is sufficient and leaves zero. Cash 106 is insufficient; the function preserves all 106 rather than charging a fee for a trade it did not accept. At the common mark 52, pre-buy wealth is 800+4*52=1008 and post-buy wealth is 693+6*52=1005. The difference is the fee, -3. This is an accounting identity, not an investment return claim.

Implementation and reproducibility

The published delayed_order.cpp is a teaching fragment. The verifier assembles a temporary translation unit by placing unchanged validated_buy.cpp before it, then a test driver after it. The compiler therefore sees AccountState and try_buy before their use. This reuses the tested primitive without duplicating its source. It is not a standalone library/header architecture; introducing that packaging is deferred. No installed build/ executable supplies today's evidence.

From the repository root run: `.venv/bin/python scripts/verify_delayed_order.py`. The result JSON records input/source hashes, each expected line and the exact counts. The verifier checks exit status, all output bytes and empty stderr on each run. It requires C++17, -Wall, -Wextra, -Wpedantic and -Werror, plus undefined-behavior instrumentation that stops on an error. Two verifier invocations compile two new executables, each executed twice.

The reference debits each share separately with Decimal, then applies the fee to a candidate balance. Hand-written expected fields must match it first. The C++ implementation reuses the existing guarded integer buy. For every trace the harness also prints the caller's original account after the call; all caller states remain unchanged, including successful traces.

Checks and observed results

All eight valid hand/reference cases match exactly. Three are filled, one unresolved, two rejected_by_model and two no_order. Decision snapshots retain initial balances, and only accepted fills charge three. All fifteen invalid integer traces produce invalid_trace with request -1, unchanged snapshots/caller and zero fee. Invalid cases cover equal/future observation, equal/early resolution, negative event index, out-of-domain account/observation/fill values, and malformed fill with no request. These are selected examples, not exhaustive testing of every integer.

Two fresh correct builds, each executed twice, produce identical JSON. Three separate scratch source mutations compile successfully and then fail the intended trace comparison: using the observed price for the debit, charging three on missing fill, and allowing equal resolution and decision events. The production fragment is unchanged. Actual Python -O execution is refused so disabled assertions cannot produce an apparent pass. Ruff lint/format pass after normal formatting; the correct implementation passed its first executable check.

Original October 23 checks passed 608 Python tests and all prior quality/C++ checks. Integration on October 24 adds make cpp-delayed-order to make check. Two fresh builds, all three mutants, -O refusal and the full suite are revalidated. No dependency was added.

Costs, failures and limitations

Cash/shares are capped at 10000; prices lie in 1..10000; quantity and fee are fixed at 2 and 3. The existing buy guard prevents unaffordable multiplication before it happens. Event indices are int values compared without arithmetic, with strict 0 <= available < decision < resolution. The C++ API accepts typed integers. It is not a raw-input parser: Boolean or fractional arguments can convert before entry. The harness rejects incompatible JSON types before writing C++ literals; a future user interface needs its own validated conversion boundary. UBSan does not replace that.

The complete-trace wrapper validates later evidence before returning any valid trace. An invalid future input invalidates the whole record; it is not a claim that an earlier real decision never occurred. The isolated decision function is the seam for L002d's future-price tests. Today's fixed cases and source shape do not prove general absence of look-ahead. We model no reservation, retry, partial execution, liquidity, spread, tax, impact or settlement. Repeated calls each create a new trace; no persistent order identity prevents duplicate real execution. This is not live trading.

Interview explanation and next step

I save the decision state, compute the request from known observation fields, and apply later fill evidence to a separate state using checked accounting. Hand values and an independent reference test the trace. Missing evidence stays unresolved; invalid chronology rejects the whole trace. Passing examples shows mechanics, not exchange fidelity, alpha or personal mastery.

Next is L002d: mutate valid future prices, preserve earlier requests, allow later accounting to differ, and detect a leaking control. L002e/f adverse fills/reproduction remain separate. C003b/C004/C005/Q011-13 and Q004a/Q005b remain gates; later strategy windows stay conditional.

October 24 integration verifies all 27 original input/artifact/QA records. The original source, results and study evidence are preserved; verifier paths now point to published files. This uses one of today's two commit slots. L002d is the next lesson, not a repeat of implementation. Personal review remains Not yet reviewed. No input blocks synthetic continuation; lawful access evidence precedes real acquisition. Stop new research after December 31.

Primary sources consulted October 23

QuantConnect, Timeslices: LEAN's information frontier illustrates exposing only data available at simulated algorithm time. Our strict event-index convention is a separate teaching choice. https://www.quantconnect.com/docs/v2/writing-algorithms/key-concepts/time-modeling/timeslices

SEC Investor.gov, Types of Orders: a last-traded price need not be the actual execution price. Our fill of 52 and fee of three are invented assumptions, not observations or broker terms. https://www.investor.gov/introduction-investing/investing-basics/how-stock-markets-work/types-orders

Microsoft Learn, Functions (C++): functions accept parameters and may return values. We use small functions with explicit inputs and return a grouped trace value. https://learn.microsoft.com/en-us/cpp/cpp/functions-cpp?view=msvc-170

Clang, UndefinedBehaviorSanitizer: -fsanitize=undefined instruments selected errors on executed paths; -fno-sanitize-recover=all stops on a detected error. It does not prove financial logic. https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

Check your understanding

1. Which state is passed by reference to try_buy, and does the caller's account change?

2. What distinguishes an absent fill from a supplied fill price of zero?

3. Why is cash 107 sufficient while 106 is not at fill 52, quantity 2 and fee 3?

4. Does today's pass establish future-price invariance or safe arbitrary text input?

Suggested answers

1. Only trace.after_resolution is passed by reference. at_decision retains the original balance. The caller's account stays unchanged because delayed_order receives initial by value; a caller must explicitly adopt the returned final state after inspecting its validity and status.

2. False has_fill means missing evidence and ignores the placeholder price. True with zero is an invalid supplied price. Missing evidence does not justify fabricating a fill or a cancellation.

3. Debit is 2*52+3=107. Equality leaves cash zero and adds two shares. One unit less produces model rejection with unchanged cash/shares and no fee; exact capacity 9998+2=10000 also passes.

4. Neither. L002d must test future-price changes and a leaking control. This C++ API uses typed integers; parsing and conversion checks must precede it. Fixed synthetic tests do not prove alpha.